OBJECTIF DU TP1 : PRISE EN MAIN

L'objectif du TP est d'une part de se familiariser avec les robots EV3 et leur API, et d'autre part de développer les éléments de base qui serviront pour les prochains TPs.



CONSIGNES

Les robots EV3 utilisés dans ce TP sont fournis montés, il est déconseillé de les démonter dans la première séance ou de les modifier. Vous pouvez commencer par vérifier les branchements suivants :

- Port C : moteur droit
- Port B : moteur gauche
- Port 2 : capteur de distance
- Port 3 : capteur couleur

Si ce n'est pas le cas veuillez bien noter les ports utilisés.

EV3 ET MICROPYTHON

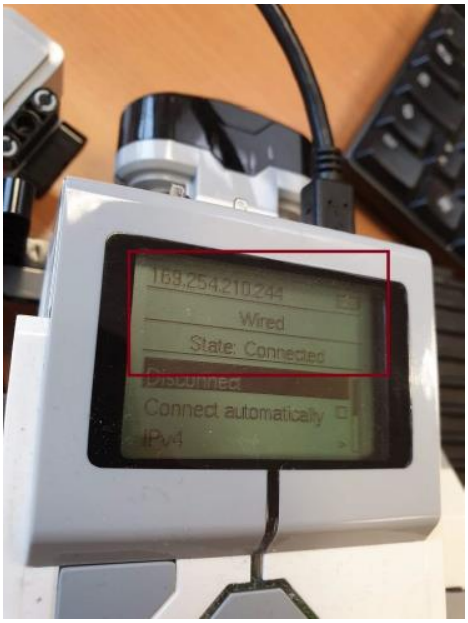
Les robots tournent sur un environnement MicroPython basé sur ev3dev et donnant accès à la librairie Pybricks exécutée sur python 3.4.0. Attention, la dernière version de python étant 3.8.5, certaines fonctionnalités accessibles sur votre ordinateur ne le seront pas sur les robots.

L'environnement MircoPython est déjà installé sur les cartes SD des robots, vous ne devez donc jamais flasher la carte SD.

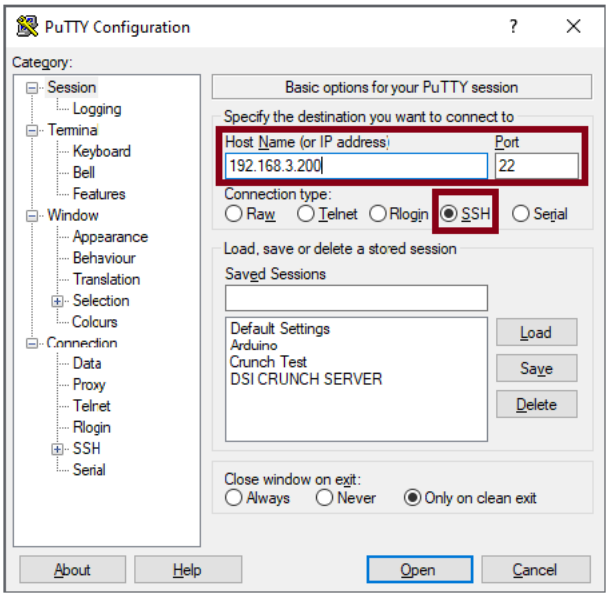
CONNEXION SSH

Pour commencer, vous allez tester la connexion avec le robot en SSH.

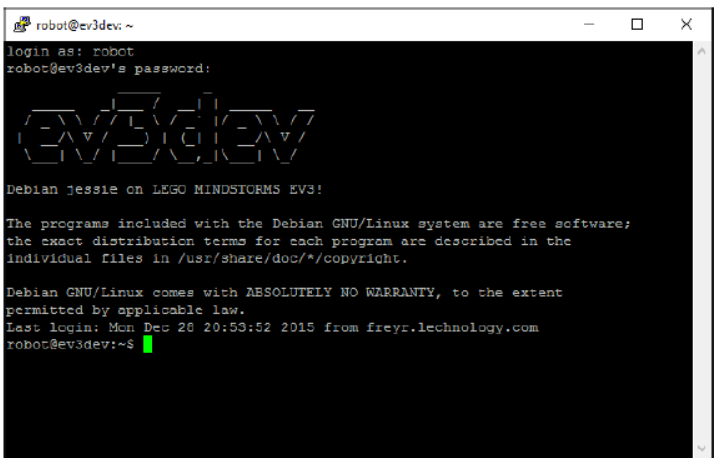
Reliez le robot à l'ordinateur grâce au câble usb. Une fois le robot connecté, celui-ci va fournir une adresse ip.



Lancez PuTTY et démarrez une connexion en utilisant l'adresse ip du robot.



Utilisez maintenant le login robot et le mot de passe maker.



Vous êtes maintenant connectés en SSH au robot.

CONNEXION WIFI

Il est maintenant conseillé de configurer le robot pour travailler en wifi de manière à éviter les désagréments de la connexion câblée usb. Vous pouvez connecter votre robot à votre smartphone ou à votre ordinateur avec un wifi en mode point d'accès.

Une fois connecté en SHH, il est possible de configurer la connexion au wifi de votre choix de la manière suivante :

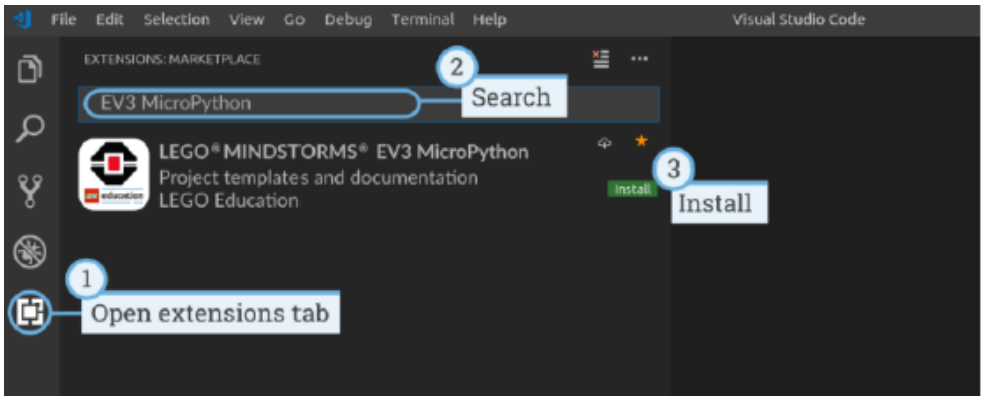
```
robot@ev3dev:~$ connmanctl
Error getting VPN connections: The name net.connman.vpn was not provided by any
connmanctl> enable wifi
Enabled wifi
connmanctl> scan wifi
Scan completed for wifi
connmanctl> services
*AO Wired               ethernet_b827ebbde13c_cable
                        wifi_e8de27077de3_hidden_managed_none
    AH04044914          wifi_e8de27077de3_41483034303434393134_managed_psk
    Frissie             wifi_e8de27077de3_46726973736965_managed_psk
    ruijgt gast         wifi_e8de27077de3_7275696a67742067617374_managed_psk
    schuur              wifi_e8de27077de3_736368757572_managed_psk
connmanctl> agent on
Agent registered
connmanctl> connect wifi_e8de27077de3_41      # You can use the TAB key at this point to autocomplete the name
connmanctl> connect wifi_e8de27077de3_41483034303434393134_managed_psk
Agent RequestInput wifi_e8de27077de3_41483034303434393134_managed_psk
  Passphrase = [ Type=psk, Requirement=mandatory ]
Passphrase? **************
Connected wifi_e8de27077de3_41483034303434393134_managed_psk
connmanctl> quit
```

Une fois le robot connecté en wifi, vous pouvez déconnecter le câble usb.

VISUAL STUDIO CODE

Visual Studio Code va permettre de coder en python et d'envoyer les programmes sur les robots.

Après avoir lancé Visual Studio Code vous pourrez installer l'extension EV3 MicroPython nécessaire au développement (voir ci-dessous).



Vous pouvez vous référer à la documentation fournie avec ce sujet :

Getting_Started_EV3_MicroPython_Lego.pdf

, que vous trouverez dans le répertoire Teams du TP.

La partie installation est déjà effectuée, vous pouvez donc commencer directement au chapitre 2.

Aussi vous pouvez vous référer au site :https://pybricks.com/ev3-micropython/startinstall.html

PREMIER PROGRAMME

Pour commencez vous aller faire un simple programme permettant d'afficher sur l'écran du robot la version Python (voir ci-dessous).

```
#!/usr/bin/env pybricks-micropython

from pybricks.tools import wait
import sys

print(sys.version)
wait(5000)
```

Remarque : tous vos programmes devront commencer par le shebang #!/usr/bin/env pybricks-micropython de manière à être dans l'environnement pybricks.

TRAVAIL DEMANDE

Vous allez développer plusieurs modules spécifiques sous forme de classes, et organisés dans plusieurs fichiers.

Remarque : les informations données par la suite ont pour but de vous aider à structurer votre base de travail qui servira tout au long du semestre, vous êtes libres de réinterpréter les indications, ainsi que d'ajouter toutes les classes et méthodes que vous jugerez nécessaires.

PILOTAGE DES MOTEURS

Créez une classe spécifique au contrôle du robot avec les méthodes de base suivantes :

• void forward(speed) : le robot avance selon la vitesse indiquée

• void rotate(angle, aSpeed) : le robot tourne sur lui-même selon l'angle et la vitesse angulaire donnés

• void stop() : le robot s'arrête

Remarque : à ce stade vous n'utiliserez que les méthodes dérivées de la classe Motor.

CAPTEUR DE DISTANCE

Créez une classe spécifique au capteur de distance avec au minimum une méthode distance() qui renvoie la distance actuelle mesurée. La classe doit pouvoir s'adapter au type du capteur, c'est-à-dire IR ou Ultrason.

CAPTEUR DE COULEUR

Créez une classe spécifique au capteur de couleur avec au minimum une méthode color() qui renvoie la couleur actuelle mesurée. Vous pouvez également commencer à faire des méthodes plus haut niveau permettant de détecter des zones au sol, identifiables par des couleurs distinctes.

ETAT DU ROBOT

Créez une classe spécifique permet de représenter l'état actuel du robot.

Vous pourrez notamment ajouter une variable distance ainsi qu'une variable couleur indiquant les dernières informations connues. Vous mettrez au fur et à mesure dans cette classe toutes les informations que vous jugerez utiles.

AFFICHEUR LCD

Créez une classe spécifique à l'affichage des informations via l'écran LCD.

Ajoutez une méthode status() permettant d'afficher l'état actuel du robot à partir de la classe précédente.

LOGS

Créez une classe spécifique permettant d'écrire des logs au format .csv.

Ajoutez une méthode log() permettant d'écrire dans un fichier l'état actuel du robot. Chaque ligne inscrite dans le fichier doit comporter une information temporelle de manière à pouvoir générer des courbes en ouvrant le fichier dans Excel.

SON ET LUMIERE

Créez une classe spécifique à l'utilisation du son et de la lumière sur la brique.

Pour les lumières il s'agit de tirer partie de la possibilité de contrôler l'affichage de couleur sur les LEDs gauche et droite. Vous pourrez donc créer des méthodes permettant de transmettre certaines informations, notamment grâce au clignotement.

PILOTAGE DES MOTEURS (HAUT-NIVEAU)

Testez la classe existante DriveBase. Vous pouvez ensuite intégrer cette classe à votre contrôleur de robot.

COMMUNICATION SANS-FIL

L'idée principale va être de connecter plusieurs robots en wifi à un point d'accès (ordinateur) de manière à faire partie d'un même réseau. L'ordinateur doit exécuter un serveur en python qui va attendre les connexions des clients (les robots). Une fois le réseau établi, les clients doivent pouvoir envoyer des messages au serveur et inversement. Un robot doit également pouvoir communiquer avec n'importe quel autre robot du réseau (en passant par le point d'accès). Chaque classe (serveur et client) aura donc au minimum une méthode send() et une méthode receive() permettant d'échanger des messages.

GESTION DES EVENEMENTS

Vous pourrez améliorer votre code de manière à intégrer une gestion des évènements. Par exemple un évènement pourrait être déclenché à chaque fois que votre module couleur détecte un changement de zone ou encore lorsqu'une limite de proximité est atteinte par le capteur de distance.

OBJECTIF DU TP2 : SUIVI DE LIGNE

L'objectif du TP est de programmer le robot afin qu'il **suive un tracé au sol**. Vous utiliserez le **capteur couleur** afin de déterminer la position du robot par rapport à la ligne. Tout ce que vous faites est dans la continuité des TPs et sera utilisé pour le futur projet.



TRAVAIL DEMANDE

Ajoutez ou complétez les classes nécessaires en vue de donner la capacité à votre robot de suivre une ligne noire sur une feuille blanche. Vous veillerez à optimiser la vitesse de déplacement et la stabilité de la trajectoire du robot. Le suivi de ligne sera réalisé grâce au PID. L'utilisation d'autres commandes plus évoluées est possible. Cependant, la commande choisie doit être bien spécifiée dans le rapport.

Le rendu du TP sera réalisé à travers un rapport, incluant le code de la commande latérale, les justificatifs des choix réalisés, ainsi que les mesures suivantes :

1. Log de l'erreur de la commande bang-bang
2. Log de l'erreur et le temps de parcours sur une ligne droite pour une distance d'1 mètre.
3. Log de l'erreur et le temps de parcours sur le tapis de course.

Les deux derniers logs de l'erreur (2 et 3) concernent seulement les erreurs engendrées par la commande latérale définitive.

COMMANDE LATERALE : PID

L'objectif de la séance de travaux pratiques est de déterminer une commande latérale du robot, lui permettant de suivre une ligne noire épaisse grâce à l'utilisation du capteur couleur. Le capteur se trouvant du côté gauche du robot, celui-ci doit suivre le bord gauche de la ligne, de manière à centrer le robot sur cette dernière. Ainsi, lorsque le capteur couleur se trouve sur le noir, le robot doit tourner à gauche, et inversement lorsqu'il détecte du blanc. Le capteur couleur sera utilisé en mode "luminosité" de manière à obtenir des valeurs entre le noir (0%) et le blanc (100%). Pour pouvoir suivre la bordure, une valeur intermédiaire doit être définie (ex. 50%). Cette valeur représente la consigne. Nous la notons x_c. Ainsi l'erreur instantanée peut être quantifiée comme suit :

$$e(t) = x(t) - x_c$$

Pour réaliser le mouvement tournant sur le robot, il suffit de jouer sur la différence des vitesses des deux roues. Sinon, il est possible d'utiliser directement la fonction drive. Celle-ci se comporte comme suit :

- drive(vitesse_d'entraînement, vitesse_de_rotation)
- Dans la vitesse_d'entraînement on envoie la vitesse du robot en mm/s

- Dans la vitesse_de_rotation on donne la vitesse de rotation désirée du robot par rapport au point situé entre les deux roues en deg/s.

COMMANDE PROPORTIONNELLE

La commande bang-bang consiste simplement à faire tourner le robot d'un angle donné δ (vitesse_de_rotation) fixé à l'avance, à droite ou à gauche selon que la valeur de l'erreur soit positive ou négative.

Dans un premier temps nous considérons que le robot doit suivre une ligne droite. Nous supposons la mise en place manuelle du robot sur la voie avec une erreur minime à l'intérieur de la ligne noire.

1- Vérifiez que la fréquence d'exécution du code est bien fixée à 100ms. Ainsi, nous avons une correction à chaque étape n.
2- En se basant sur la commande bang-bang, définir la vitesse du robot et l'angle δ, lui permettant de suivre la voie sans diverger, à savoir sans oscillations croissantes (en cas d'échec, il est possible de passer à la question suivante).
3- Introduire la commande proportionnelle en maintenant la vitesse choisie précédemment et donner la valeur de k_p permettant si possible d'éliminer les oscillations.
   a. S'inspirer de la commande bang-bang, comme présenté dans le cours, pour le calcul de k_p :

$$\delta(n) = k_p e(n).$$

COMMANDE PI

Nous considérons le suivi d'une ligne avec des courbures. La présence de courbure peut engendrer un éloignement important du robot de la trajectoire voulue, ce qui nécessite l'introduction de la commande à base de l'intégrale de l'erreur. Pour réaliser cette partie vous pouvez utiliser le tapis de compétition :

4- Vérifier si k_p tout est suffisant pour suivre la courbure.
5- Définir l'angle de rotation de la manière suivante :

$$\delta(n) = k_p e(n) + k_i \tau \sum_{i=0}^{k} e(n)$$ [1]

   a. Vérifier que l'erreur ne dépasse pas les bornes de la commande que vous vous êtes fixées (ex. -50, 50). Nous notons les bornes $\mp\overline{\delta}$ (à ne pas confondre avec celle précédemment calculée pour la commande bang-bang).
   b. Borner l'intégrale de l'erreur de sorte que :

$$\left| k_i \tau \sum_{i=0}^{k} e(n) \right| \leq \overline{\delta}$$

   c. Faire varier le k_i à travers une approche croissante pour permettre le retour du robot vers la ligne.

COMMANDE PID

Vous remarquez que le robot peut parfois sortir complètement de la voie. Ceci est dû à l'overshoot généré par l'intégrale de l'erreur. Une première approche consiste à augmenter la valeur du k_p et à diminuer celle de k_i. Néanmoins, ceci risque de générer des oscillations de nouveau. Pour pallier ce problème, nous introduisons à la commande PI initiale, la dérivée de l'erreur.

6- Définir le steering de la manière suivante :

[1] Il est possible d'utiliser directement $k'_i = k_i \tau$ mais il faut le mentionner.

$$\boldsymbol{\delta(n) = k_p e(n) + k_i \tau \sum_{i=0}^{k} e(n) + \frac{k_d}{\tau}\big(e(n) - e(n-1)\big)}$$ [2]

d. Faire varier le k_d à travers une approche croissante pour permettre le retour du robot vers la ligne. La valeur de $\boldsymbol{k_d}$ doit être faible notamment à cause de la division.
e. Améliorer les résultats obtenus : Faire varier les trois gains selon le résultat observé, réguler la vitesse du robot en fonction de l'erreur, ajuster, si nécessaire, le k_i en fonction de l'orientation de l'erreur (à droite, à gauche).

TESTS

Le rendu du TP sera réalisé à travers un rapport, incluant le code de la commande latérale ainsi que les mesures suivantes :

1. Log de l'erreur de la commande bang-bang
2. Log de l'erreur et le temps de parcours sur une ligne droite
3. Log de l'erreur et le temps de parcours sur le tapis de course

Une compétition entre les différents groupes sur le tapis de course sera réalisée à la fin du TP.

[2] Il est possible d'utiliser directement $k'_d = k_d/\tau$ mais il faut le mentionner